Mémoire: QUIC: A UDP-Based Multiplexed and Secure Transport draft-ietf-quic-transport-29

Academic year : 2020 - 2021

Teacher : Bonaventure Olivier,
Legay Axel
Course : Mémoire
Collaborators :
Crochet Christophe,
Jean-François Sambon

1 Streams

- Streams are identified within a connection by a numeric value, referred to as the stream ID. A stream ID is a 62-bit integer (0 to 2ĝ2-1) that is unique for all streams on a connection.

Bits	Stream Type
0x0	Client-Initiated, Bidirectional
0x1	Server-Initiated, Bidirectional
0x2	Client-Initiated, Unidirectional
0x3	Server-Initiated, Unidirectional

Table 1: Stream ID Types

2 Variable-Length Integer Encoding

QUIC packets and frames commonly use a variable-length encoding for non-negative integer values. This encoding ensures that smaller integer values need fewer bytes to encode.

The QUIC variable-length integer encoding reserves the two most significant bits of the first byte to encode the base 2 logarithm of the integer encoding length in bytes. The integer value is encoded on the remaining bits, in network byte order.

This means that integers are encoded on 1, 2, 4, or 8 bytes and can encode 6, 14, 30, or 62 bit values respectively. Table 4 summarizes the encoding properties.

2Bit	Length	Usable Bits	Range
00	1	6	0-63
01	2	14	0-16383
10	4	30	0-1073741823
11	8	62	0-4611686018427387903

Table 4: Summary of Integer Encodings

3 Packets

3.1 Long Header Packets

```
Long Header Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2),
  Type-Specific Bits (4),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
}
```

Figure 9: Long Header Packet Format

Type	Name	Section
0x0	Initial	Section 17.2.2
0x1	0-RTT	Section 17.2.3
0x2	Handshake	Section 17.2.4
0x3	Retry	Section 17.2.5

Table 5: Long Header Packet Types

3.1.1 Version Negotiation Packet

The Version field of a Version Negotiation packet MUST be set to 0x00000000.

```
Version Negotiation Packet {
  Header Form (1) = 1,
  Unused (7),
  Version (32) = 0,
  Destination Connection ID Length (8),
  Destination Connection ID (0..2040),
  Source Connection ID Length (8),
  Source Connection ID (0..2040),
  Supported Version (32) ...,
}
```

Figure 10: Version Negotiation Packet

3.1.2 Initial Packet

```
Initial Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 0,
  Reserved Bits (2),
  Packet Number Length (2),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Token Length (i),
  Token (..),
  Length (i),
  Packet Number (8..32),
  Packet Payload (..),
}
```

Figure 11: Initial Packet

3.1.3 0-RTT

```
0-RTT Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 1,
  Reserved Bits (2),
  Packet Number Length (2),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
```

```
  Source Connection ID (0..160),
  Length (i),
  Packet Number (8..32),
  Packet Payload (..),
}
```

Figure 12: 0-RTT Packet

3.1.4 Handshake Packet

```
Handshake Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 2,
  Reserved Bits (2),
  Packet Number Length (2),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Length (i),
  Packet Number (8..32),
  Packet Payload (..),
}
```

Figure 13: Handshake Protected Packet

3.1.5 Retry Packet

```
Retry Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 3,
  Unused (4),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Retry Token (..),
  Retry Integrity Tag (128),
}
```

Figure 14: Retry Packet

3.2 Short Header Packets

```
Short Header Packet {
  Header Form (1) = 0,
```

```
  Fixed Bit (1) = 1,
  Spin Bit (1),
  Reserved Bits (2),
  Key Phase (1),
  Packet Number Length (2),
  Destination Connection ID (0..160),
  Packet Number (8..32),
  Packet Payload (..),
}
```

Figure 15: Short Header Packet Format

3.3 Stateless Reset Packet

```
Stateless Reset {
  Fixed Bits (2) = 1,
  Unpredictable Bits (38..),
  Stateless Reset Token (128),
}
```

Figure 6: Stateless Reset Packet

4 Transport Parameter Encoding

```
Transport Parameters {
  Transport Parameter (..) ...,
}
```

Figure 16: Sequence of Transport Parameters

```
Transport Parameter {
  Transport Parameter ID (i),
  Transport Parameter Length (i),
  Transport Parameter Value (..),
}
```

Figure 17: Transport Parameter Encoding

```
 original_destination_connection_id (0x00):  The value of the
    Destination Connection ID field from the first Initial packet sent
    by the client; see Section 7.3.  This transport parameter is only
    sent by a server.

max_idle_timeout (0x01):  The max idle timeout is a value in
    milliseconds that is encoded as an integer; see (Section 10.2).
    Idle timeout is disabled when both endpoints omit this transport
    parameter or specify a value of 0.
```

stateless_reset_token (0x02): A stateless reset token is used in verifying a stateless reset; see Section 10.4. This parameter is a sequence of 16 bytes. This transport parameter MUST NOT be sent by a client, but MAY be sent by a server. A server that does not send this transport parameter cannot use stateless reset (Section 10.4) for the connection ID negotiated during the handshake

max_udp_payload_size (0x03): The maximum UDP payload size parameter is an integer value that limits the size of UDP payloads that the endpoint is willing to receive. UDP packets with payloads larger than this limit are not likely to be processed by the receiver.

The default for this parameter is the maximum permitted UDP payload of 65527. Values below 1200 are invalid.

This limit does act as an additional constraint on datagram size in the same way as the path MTU, but it is a property of the endpoint and not the path; see Section 14. It is expected that this is the space an endpoint dedicates to holding incoming packets.

initial_max_data (0x04): The initial maximum data parameter is an integer value that contains the initial value for the maximum amount of data that can be sent on the connection. This is equivalent to sending a MAX_DATA (Section 19.9) for the connection immediately after completing the handshake.

initial_max_stream_data_bidi_local (0x05): This parameter is an integer value specifying the initial flow control limit for locally-initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that sends the transport parameter. In client transport parameters, this applies to streams with an identifier with the least significant two bits set to 0x0; in server transport parameters, this applies to streams with the least significant two bits set to 0x1.

initial_max_stream_data_bidi_remote (0x06): This parameter is an integer value specifying the initial flow control limit for peer-initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that receives the transport parameter. In client transport parameters, this applies to streams with an identifier with the least significant two bits set to 0x1; in server transport parameters, this applies to streams with the least significant two bits set to 0x0.

initial_max_stream_data_uni (0x07): This parameter is an integer value specifying the initial flow control limit for unidirectional streams. This limit applies to newly created unidirectional

streams opened by the endpoint that receives the transport parameter. In client transport parameters, this applies to

streams with an identifier with the least significant two bits set to 0x3; in server transport parameters, this applies to streams with the least significant two bits set to 0x2.

initial_max_streams_bidi (0x08): The initial maximum bidirectional streams parameter is an integer value that contains the initial maximum number of bidirectional streams the peer may initiate. If this parameter is absent or zero, the peer cannot open bidirectional streams until a MAX_STREAMS frame is sent. Setting this parameter is equivalent to sending a MAX_STREAMS (Section 19.11) of the corresponding type with the same value.

initial_max_streams_uni (0x09): The initial maximum unidirectional streams parameter is an integer value that contains the initial maximum number of unidirectional streams the peer may initiate. If this parameter is absent or zero, the peer cannot open unidirectional streams until a MAX_STREAMS frame is sent. Setting this parameter is equivalent to sending a MAX_STREAMS (Section 19.11) of the corresponding type with the same value.

ack_delay_exponent (0x0a): The ACK delay exponent is an integer value indicating an exponent used to decode the ACK Delay field in the ACK frame (Section 19.3). If this value is absent, a default value of 3 is assumed (indicating a multiplier of 8). Values above 20 are invalid.

max_ack_delay (0x0b): The maximum ACK delay is an integer value indicating the maximum amount of time in milliseconds by which the endpoint will delay sending acknowledgments. This value SHOULD include the receiver's expected delays in alarms firing. For example, if a receiver sets a timer for 5ms and alarms commonly fire up to 1ms late, then it should send a max_ack_delay of 6ms. If this value is absent, a default of 25 milliseconds is assumed. Values of 2^14 or greater are invalid.

disable_active_migration (0x0c): The disable active migration transport parameter is included if the endpoint does not support active connection migration (Section 9) on the address being used during the handshake. When a peer sets this transport parameter, an endpoint MUST NOT use a new local address when sending to the address that the peer used during the handshake. This transport parameter does not prohibit connection migration after a client has acted on a preferred_address transport parameter. This parameter is a zero-length value.

preferred_address (0x0d): The server's preferred address is used to effect a change in server address at the end of the handshake, as described in Section 9.6. The format of this transport parameter

is shown in Figure 21. This transport parameter is only sent by a server. Servers MAY choose to only send a preferred address of one address family by sending an all-zero address and port (0.0.0.0:0 or ::.0) for the other family. IP addresses are encoded in network byte order.

The Connection ID field and the Stateless Reset Token field contain an alternative connection ID that has a sequence number of 1; see Section 5.1.1. Having these values bundled with the preferred address ensures that there will be at least one unused active connection ID when the client initiates migration to the preferred address.

The Connection ID and Stateless Reset Token fields of a preferred address are identical in syntax and semantics to the corresponding fields of a NEW_CONNECTION_ID frame (Section 19.15). A server that chooses a zero-length connection ID MUST NOT provide a preferred address. Similarly, a server MUST NOT include a zero-length connection ID in this transport parameter. A client MUST treat violation of these requirements as a connection error of type TRANSPORT_PARAMETER_ERROR.

```
Preferred Address {
  IPv4 Address (32),
  IPv4 Port (16),
  IPv6 Address (128),
  IPv6 Port (16),
  CID Length (8),
  Connection ID (..),
  Stateless Reset Token (128),
}
```

Figure 21: Preferred Address format

active_connection_id_limit (0x0e): The active connection ID limit is an integer value specifying the maximum number of connection IDs from the peer that an endpoint is willing to store. This value includes the connection ID received during the handshake, that received in the preferred_address transport parameter, and those received in NEW_CONNECTION_ID frames. The value of the active_connection_id_limit parameter MUST be at least 2. An endpoint that receives a value less than 2 MUST close the connection with an error of type TRANSPORT_PARAMETER_ERROR. If this transport parameter is absent, a default of 2 is assumed. If an endpoint issues a zero-length connection ID, it will never send a NEW_CONNECTION_ID frame and therefore ignores the active_connection_id_limit value received from its peer.

initial_source_connection_id (0x0f): The value that the endpoint included in the Source Connection ID field of the first Initial packet it sends for the connection; see Section 7.3.

retry_source_connection_id (0x10): The value that the server included in the Source Connection ID field of a Retry packet; see Section 7.3. This transport parameter is only sent by a server.

If present, transport parameters that set initial flow control limits (initial_max_stream_data_bidi_local, initial_max_stream_data_bidi_remote, and initial_max_stream_data_uni) are equivalent to sending a MAX_STREAM_DATA frame (Section 19.10) on every stream of the corresponding type immediately after opening. If the transport parameter is absent, streams of that type start with a flow control limit of 0.

A client MUST NOT include any server-only transport parameter: original_destination_connection_id, preferred_address, retry_source_connection_id, or stateless_reset_token. A server MUST treat receipt of any of these transport parameters as a connection error of type TRANSPORT_PARAMETER_ERROR.

5 Frames

```
Packet Payload {
  Frame (..) ...,
}
```

Figure 7: QUIC Payload

```
Frame {
  Frame Type (i),
  Type-Dependent Fields (..),
}
```

Figure 8: Generic Frame Layout

Type Value	Frame Type Name	Definition	Packets
0x00	PADDING	Section 19.1	IH01
0x01	PING	Section 19.2	IH01
0x02 - 0x03	ACK	Section 19.3	IH_1
0x04	RESET_STREAM	Section 19.4	__01
0x05	STOP_SENDING	Section 19.5	__01

0x06	CRYPTO	Section 19.6	IH_1
0x07	NEW_TOKEN	Section 19.7	___1
0x08 - 0x0f	STREAM	Section 19.8	__01
0x10	MAX_DATA	Section 19.9	__01
0x11	MAX_STREAM_DATA	Section 19.10	__01
0x12 - 0x13	MAX_STREAMS	Section 19.11	__01
0x14	DATA_BLOCKED	Section 19.12	__01
0x15	STREAM_DATA_BLOCKED	Section 19.13	__01
0x16 - 0x17	STREAMS_BLOCKED	Section 19.14	__01
0x18	NEW_CONNECTION_ID	Section 19.15	__01
0x19	RETIRE_CONNECTION_ID	Section 19.16	__01
0x1a	PATH_CHALLENGE	Section 19.17	__01
0x1b	PATH_RESPONSE	Section 19.18	__01
0x1c - 0x1d	CONNECTION_CLOSE	Section 19.19	ih01
0x1e	HANDSHAKE_DONE	Section 19.20	___1

Table 3: Frame Types

I: Initial (Section 17.2.2)

H: Handshake (Section 17.2.4)

0: 0-RTT (Section 17.2.3)

1: 1-RTT (Section 17.3)

*: A CONNECTION_CLOSE frame of type 0x1c can appear in Initial, Handshake, and 1-RTT packets, whereas a CONNECTION_CLOSE of type 0x1d can only appear in a 1-RTT packet.

5.1 PADDING Frames

```
PADDING Frame {
  Type (i) = 0x00,
}
```

5.2 Ping Frames

```
PING Frame {
  Type (i) = 0x01,
}
```

5.3 ACK Frames

```
ACK Frame {
  Type (i) = 0x02..0x03,
  Largest Acknowledged (i),
  ACK Delay (i),
  ACK Range Count (i),
  First ACK Range (i),
  ACK Range (..) ...,
  [ECN Counts (..)],
}

ACK Range {
  Gap (i),
  ACK Range Length (i),
}
```

Figure 20: ACK Range

```
ECN Counts {
  ECT0 Count (i),
  ECT1 Count (i),
  ECN-CE Count (i),
}
```

Figure 21: ECN Count Format

5.3.1 RESET_STREAM Frame

```
RESET_STREAM Frame {
  Type (i) = 0x04,
  Stream ID (i),
  Application Protocol Error Code (i),
  Final Size (i),
}
```

Figure 22: RESET_STREAM Frame Format

5.3.2 STOP_SENDING Frame

```
STOP_SENDING Frame {
  Type (i) = 0x05,
  Stream ID (i),
  Application Protocol Error Code (i),
}
```

Figure 23: STOP_SENDING Frame Format

5.3.3 CRYPTO Frames

```
CRYPTO Frame {
  Type (i) = 0x06,
  Offset (i),
  Length (i),
  Crypto Data (..),
}
```

Figure 24: CRYPTO Frame Format

5.3.4 NEW_TOKEN Frame

```
NEW_TOKEN Frame {
  Type (i) = 0x07,
  Token Length (i),
  Token (..),
}
```

Figure 25: NEW_TOKEN Frame Format

5.3.5 STREAM Frames

```
STREAM Frame {
  Type (i) = 0x08..0x0f,
  Stream ID (i),
  [Offset (i)],
  [Length (i)],
  Stream Data (..),
}
```

Figure 26: STREAM Frame Format

5.3.6 MAX_DATA Frame

```
MAX_DATA Frame {
  Type (i) = 0x10,
  Maximum Data (i),
}
```

Figure 27: MAX_DATA Frame Format

5.3.7 MAX_STREAM_DATA Frame

```
MAX_STREAM_DATA Frame {
  Type (i) = 0x11,
  Stream ID (i),
  Maximum Stream Data (i),
}
```

Figure 28: MAX_STREAM_DATA Frame Format

5.3.8 MAX_STREAMS Frames

```
MAX_STREAMS Frame {
  Type (i) = 0x12..0x13,
  Maximum Streams (i),
}
```

Figure 29: MAX_STREAMS Frame Format

5.3.9 DATA_BLOCKED Frame

```
DATA_BLOCKED Frame {
  Type (i) = 0x14,
  Maximum Data (i),
}
```

Figure 30: DATA_BLOCKED Frame Format

5.3.10 STREAM_DATA_BLOCKED Frame

```
STREAM_DATA_BLOCKED Frame {
  Type (i) = 0x15,
  Stream ID (i),
  Maximum Stream Data (i),
}
```

Figure 31: STREAM_DATA_BLOCKED Frame Format

5.3.11 STREAMS_BLOCKED Frames

```
STREAMS_BLOCKED Frame {
  Type (i) = 0x16..0x17,
  Maximum Streams (i),
}
```

Figure 32: STREAMS_BLOCKED Frame Format

5.3.12 NEW_CONNECTION_ID Frame

```
NEW_CONNECTION_ID Frame {
  Type (i) = 0x18,
  Sequence Number (i),
  Retire Prior To (i),
  Length (8),
  Connection ID (8..160),
  Stateless Reset Token (128),
}
```

Figure 33: NEW_CONNECTION_ID Frame Format

5.3.13 RETIRE_CONNECTION_ID Frame

```
RETIRE_CONNECTION_ID Frame {
  Type (i) = 0x19,
  Sequence Number (i),
}
```

Figure 34: RETIRE_CONNECTION_ID Frame Format

5.3.14 PATH_CHALLENGE Frame

```
PATH_CHALLENGE Frame {
  Type (i) = 0x1a,
  Data (64),
}

PATH_RESPONSE Frame {
  Type (i) = 0x1b,
  Data (64),
}
```

Figure 35: PATH_CHALLENGE Frame Format

5.3.15 CONNECTION_CLOSE Frames

```
CONNECTION_CLOSE Frame {
  Type (i) = 0x1c..0x1d,
  Error Code (i),
  [Frame Type (i)],
  Reason Phrase Length (i),
  Reason Phrase (..),
}
```

Figure 36: CONNECTION_CLOSE Frame Format

6 Transport Error Codes

QUIC error codes are 62-bit unsigned integers. This section lists the defined QUIC transport error codes that may be used in a CONNECTION_CLOSE frame. These errors apply to the entire connection.

```
CONNECTION_REFUSED (0x2):  The server refused to accept a new
   connection.

FLOW_CONTROL_ERROR (0x3):  An endpoint received more data than it
   permitted in its advertised data limits; see Section 4.

STREAM_LIMIT_ERROR (0x4):  An endpoint received a frame for a stream
```

identifier that exceeded its advertised stream limit for the corresponding stream type.

STREAM_STATE_ERROR (0x5): An endpoint received a frame for a stream that was not in a state that permitted that frame; see Section 3.

FINAL_SIZE_ERROR (0x6): An endpoint received a STREAM frame containing data that exceeded the previously established final size. Or an endpoint received a STREAM frame or a RESET_STREAM frame containing a final size that was lower than the size of stream data that was already received. Or an endpoint received a STREAM frame or a RESET_STREAM frame containing a different final size to the one already established.

FRAME_ENCODING_ERROR (0x7): An endpoint received a frame that was badly formatted. For instance, a frame of an unknown type, or an ACK frame that has more acknowledgment ranges than the remainder of the packet could carry.

TRANSPORT_PARAMETER_ERROR (0x8): An endpoint received transport parameters that were badly formatted, included an invalid value, was absent even though it is mandatory, was present though it is forbidden, or is otherwise in error.

CONNECTION_ID_LIMIT_ERROR (0x9): The number of connection IDs provided by the peer exceeds the advertised active_connection_id_limit.

PROTOCOL_VIOLATION (0xA): An endpoint detected an error with protocol compliance that was not covered by more specific error codes.

INVALID_TOKEN (0xB): A server received a Retry Token in a client Initial that is invalid.

APPLICATION_ERROR (0xC): The application or application protocol caused the connection to be closed.

CRYPTO_BUFFER_EXCEEDED (0xD): An endpoint has received more data in CRYPTO frames than it can buffer.

CRYPTO_ERROR (0x1XX): The cryptographic handshake failed. A range of 256 values is reserved for carrying error codes specific to the cryptographic handshake that is used. Codes for errors occurring when TLS is used for the crypto handshake are described in Section 4.8 of [QUIC-TLS].

7 QUIC TLS

7.1 Carrying TLS Messages

Packet Type	Encryption Keys	PN Space
Initial	Initial secrets	Initial
0-RTT Protected	0-RTT	Application data
Handshake	Handshake	Handshake
Retry	Retry	N/A
Version Negotiation	N/A	N/A
Short Header	1-RTT	Application data

Table 1: Encryption Keys by Packet Type

7.2 Header Protection

```
mask = header_protection(hp_key, sample)

pn_length = (packet[0] & 0x03) + 1
if (packet[0] & 0x80) == 0x80:
   # Long header: 4 bits masked
   packet[0] ^= mask[0] & 0x0f
else:
   # Short header: 5 bits masked
   packet[0] ^= mask[0] & 0x1f

# pn_offset is the start of the Packet Number field.
packet[pn_offset:pn_offset+pn_length] ^= mask[1:1+pn_length]
```

Figure 6: Header Protection Pseudocode

Figure 7 shows the protected fields of long and short headers marked with an E. Figure 7 also shows the sampled fields.

```
Initial Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 0,
  Reserved Bits (2),         # Protected
  Packet Number Length (2),  # Protected
  Version (32),
```

```
    DCID Len (8),
    Destination Connection ID (0..160),
    SCID Len (8),
    Source Connection ID (0..160),
    Token Length (i),
    Token (..),
    Length (i),
    Packet Number (8..32),      # Protected
    Protected Payload (0..24), # Skipped Part
    Protected Payload (128),   # Sampled Part
    Protected Payload (..)     # Remainder
  }

  Short Header Packet {
    Header Form (1) = 0,
    Fixed Bit (1) = 1,
    Spin Bit (1),
    Reserved Bits (2),         # Protected
    Key Phase (1),             # Protected
    Packet Number Length (2),  # Protected
    Destination Connection ID (0..160),
    Packet Number (8..32),     # Protected
    Protected Payload (0..24), # Skipped Part
    Protected Payload (128),   # Sampled Part
    Protected Payload (..),    # Remainder
  }
```

Figure 7: Header Protection and Ciphertext Sample

The sampled ciphertext for a packet with a short header can be determined by the following pseudocode:

```
sample_offset = 1 + len(connection_id) + 4

sample = packet[sample_offset..sample_offset+sample_length]

sample_offset = 7 + len(destination_connection_id) +
                    len(source_connection_id) +
                    len(payload_length) + 4
if packet_type == Initial:
    sample_offset += len(token_length) +
                     len(token)

sample = packet[sample_offset..sample_offset+sample_length]
```

7.3 Retry Packet Integrity

The Retry Integrity Tag is a 128-bit field that is computed as the output of AEAD_AES_128_GCM [AEAD] used with the following inputs:

```
*  The secret key, K, is 128 bits equal to
   0xccce187ed09a09d05728155a6cb96be1.

*  The nonce, N, is 96 bits equal to 0xe54930f97f2136f0530a8c1c.

*  The plaintext, P, is empty.

*  The associated data, A, is the contents of the Retry Pseudo-
   Packet, as illustrated in Figure 8:

The secret key and the nonce are values derived by calling HKDF-
Expand-Label using
0x8b0d37eb8535022ebc8d76a207d80df22646ec06dc809642c30a8baa2baaff4c as
the secret, with labels being "quic key" and "quic iv" (Section 5.1).

Retry Pseudo-Packet {
  ODCID Length (8),
  Original Destination Connection ID (0..160),
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 3,
  Type-Specific Bits (4),
  Version (32),
  DCID Len (8),
  Destination Connection ID (0..160),
  SCID Len (8),
  Retry Token (..),
}
```

Figure 8: Retry Pseudo-Packet